
Documentação de Projeto

para o sistema

DiceBox

Versão 1.0

Projeto de sistema elaborado pelo aluno Vinícius Simões Viana
como parte da disciplina **Projeto de Software**.

07 de junho, 2026

Tabela de Conteúdo

1. Introdução	1
2. Modelos de Usuário e Requisitos	1
2.1 Descrição de Atores	1
2.2 Modelo de Casos de Uso	1
2.3 Diagrama de Sequência do Sistema	3
3. Modelos de Projeto	12
3.1 Arquitetura	12
3.2 Diagrama de Componentes e Implantação.	13
3.3 Diagrama de Classes	15
3.4 Diagramas de Sequência	16
3.5 Diagramas de Comunicação	18
3.6 Diagramas de Estados	20
4. Modelos de Dados	21

Histórico de Revisões

Nome	Data	Razões para Mudança	Versão
Descrição de cenário	30/05	Início do desenvolvimento do projeto	0.1
Arquitetura	03/06	Descrevendo escolhas arquiteturais do projeto	0.2
DC e DI	04/06	Desenvolvimento do Diagrama de Componente e do Diagrama de Implantação	0.3
Diag. Estado / Classe / Seção Dados	05/06	Desenvolvidos o Diagrama de Estado, Diagrama de Classe e descrevi a seção de Modelos de Dados	0.4
DER, Diag. Comunicação	06/06	Desenvolvido Diagrama Entidade-Relacionamento usando notação Pé de Galinha para completar a seção Modelos de Dados; Desenhado Diagrama de Comunicação	0.5
Diags. Sequência	06/06	Desenhado um Diagrama de Sequência por caso de uso	0.6
Revisão do Material + Release	07/06	Organização por tópicos nas seções; Entrega Final	1.0

1. Introdução

Este documento agrega: 1) a elaboração e revisão de modelos de domínio e 2) modelos de projeto para o sistema DiceBox. A referência principal para a descrição geral do problema, domínio e requisitos do sistema é o documento de especificação que descreve a visão de domínio do sistema.

2. Modelos de Usuário e Requisitos

2.1 Descrição de Atores

Usuário: Ator genérico que representa qualquer pessoa com acesso ao sistema.

Cliente: Herda de Usuário. O cliente poderá gerenciar sua conta, navegar pelo catálogo de jogos, gerenciar seu carrinho de compras, realizar pedidos, realizar pagamentos, acompanhar pedidos e avaliar suas compras.

Vendedor: Herda de Usuário. O vendedor poderá gerenciar sua conta, gerenciar seus produtos à venda, controlar seu estoque e gerenciar pedidos (incluindo as ações de realizar envios e notificar status).

API de Pagamento: Sistema externo responsável por processar as transações financeiras.

2.2 Modelo de Casos de Uso

Nesta subseção é apresentado o diagrama de casos de uso do sistema. Para facilitar o rastreo e as referências no restante do documento, cada ação foi mapeada com um ID exclusivo, listados abaixo:

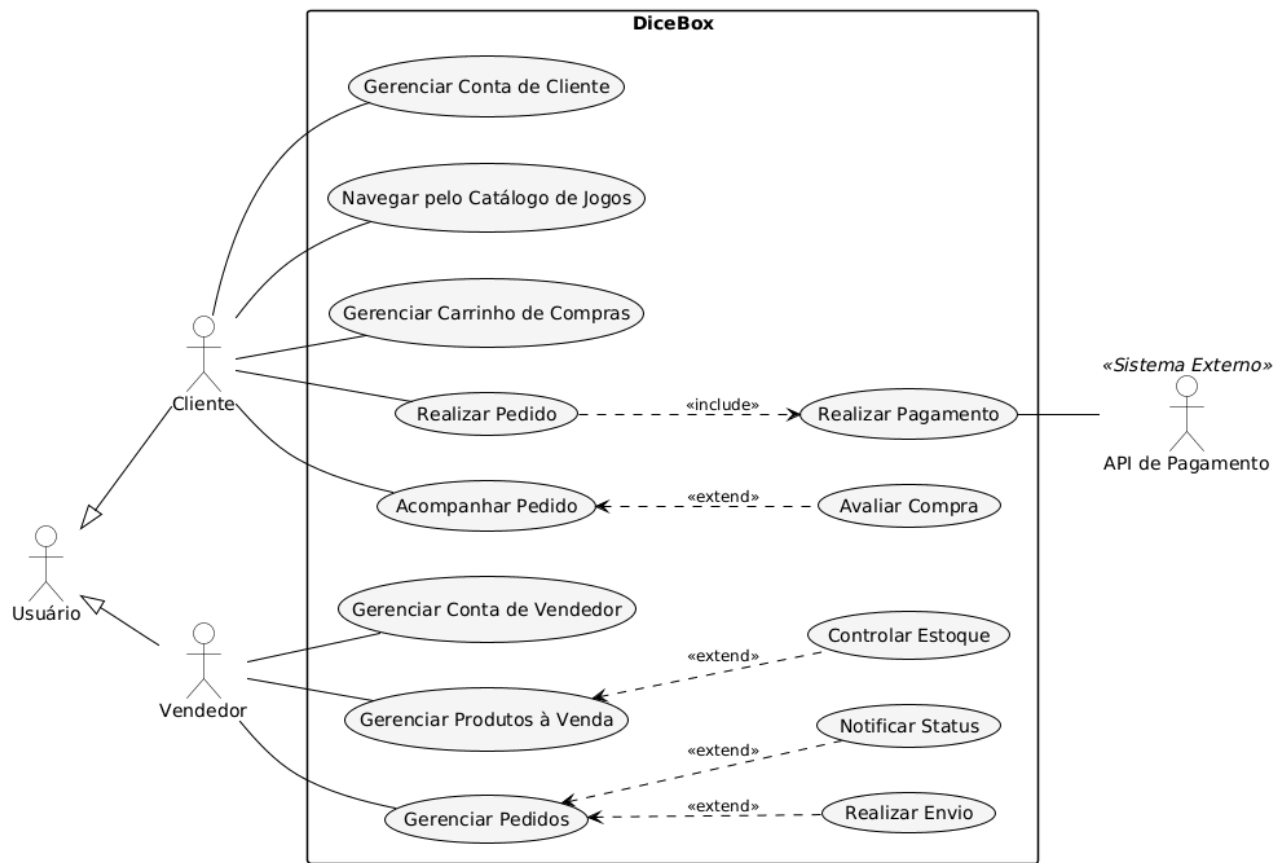
Fluxo do Cliente

- UC01 - Gerenciar Conta de Cliente
- UC02 - Navegar pelo Catálogo de Jogos
- UC03 - Gerenciar Carrinho de Compras
- UC04 - Realizar Pedido
- UC05 - Realizar Pagamento
- UC06 - Acompanhar Pedido
- UC07 - Avaliar Compra

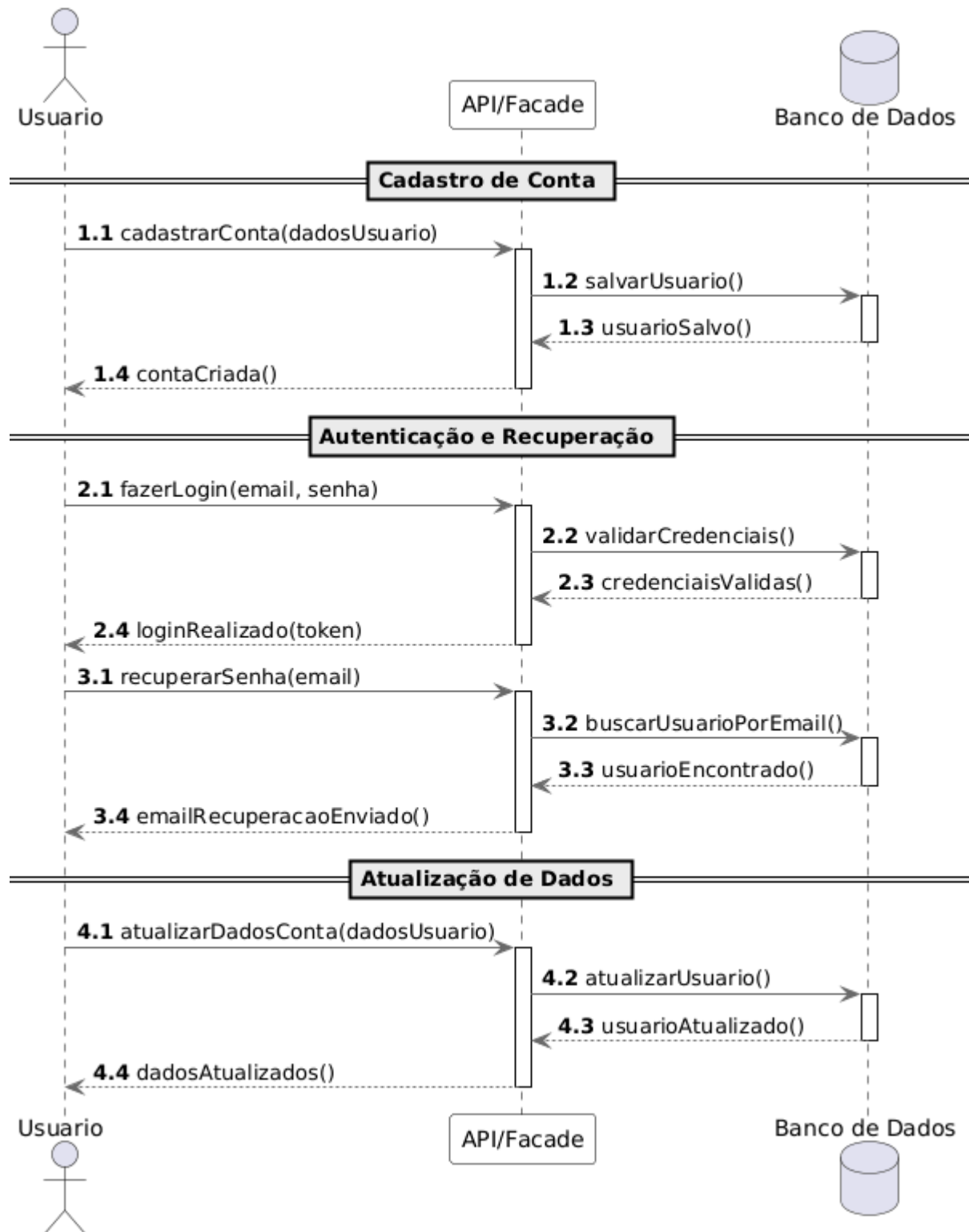
Fluxo do Vendedor

- UC08 - Gerenciar Conta de Vendedor
- UC09 - Gerenciar Produtos à Venda
- UC10 - Gerenciar Pedidos
- UC11 - Controlar Estoque
- UC12 - Realizar Envio

- **UC13** - Notificar Status



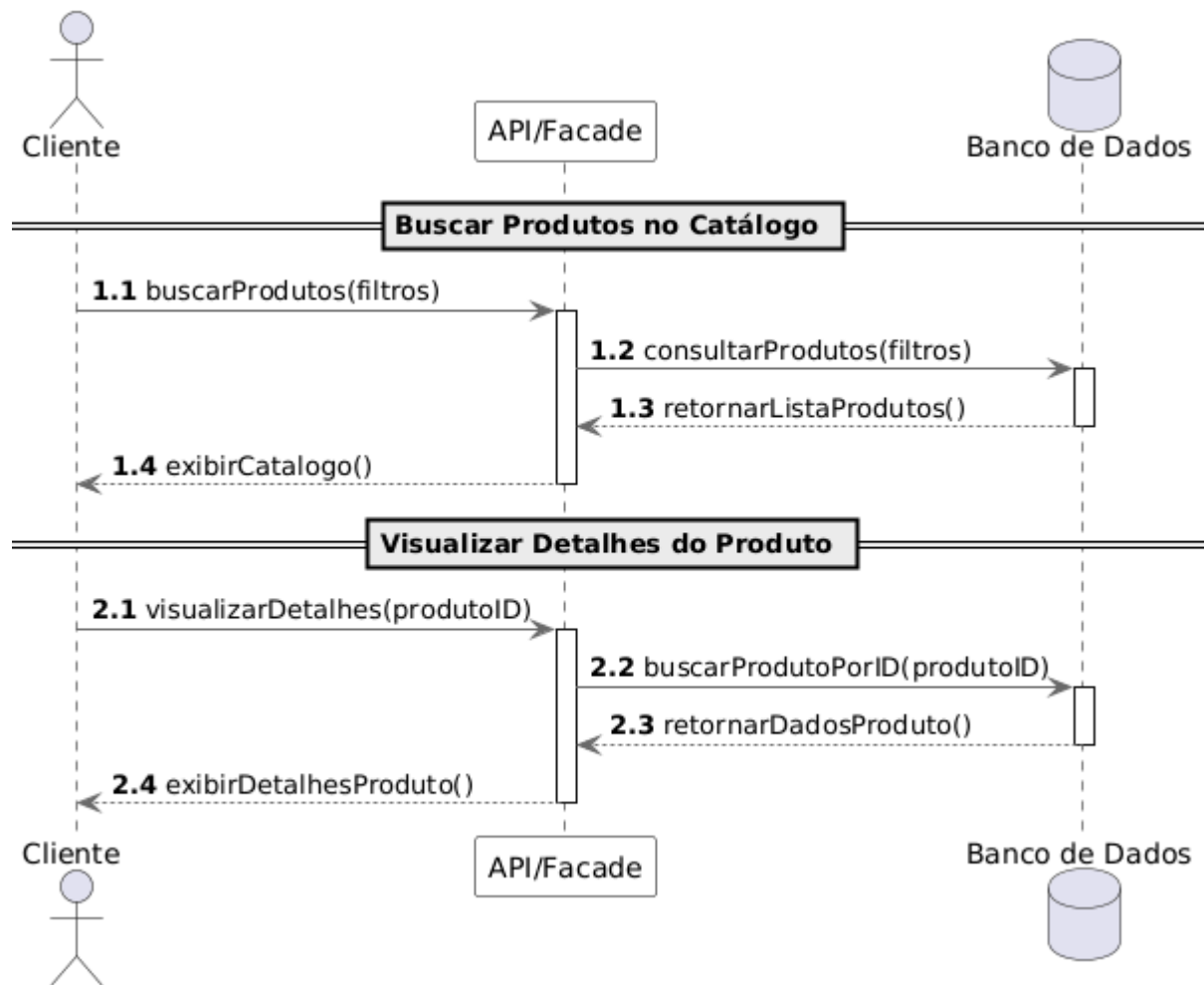
2.3 Diagrama de Sequência do Sistema



Contrato	CO-01: Cadastrar Conta
Operação	cadastrarConta(dadosUsuario)
Referências cruzadas	Caso de Uso: Gerenciar Conta (Cliente ou Vendedor)
Pré-condições	Os dados informados (como e-mail, CPF/CNPJ) não devem existir previamente no Banco de Dados.
Pós-condições	- Uma nova instância de Usuario (Cliente ou Vendedor) foi criada. - Os atributos da instância foram inicializados com base em dadosUsuario. - A nova instância de Usuario foi salva e persistida no Banco de Dados.

Contrato	CO-02: Fazer Login
Operação	fazerLogin(email, senha)
Referências cruzadas	Caso de Uso: Gerenciar Conta (Cliente ou Vendedor)
Pré-condições	O e-mail deve estar cadastrado no Banco de Dados e a senha fornecida deve ser válida.
Pós-condições	- A identidade do Usuario foi verificada com sucesso no Banco de Dados. - Um token de sessão/acesso foi gerado e retornado para o Usuario.

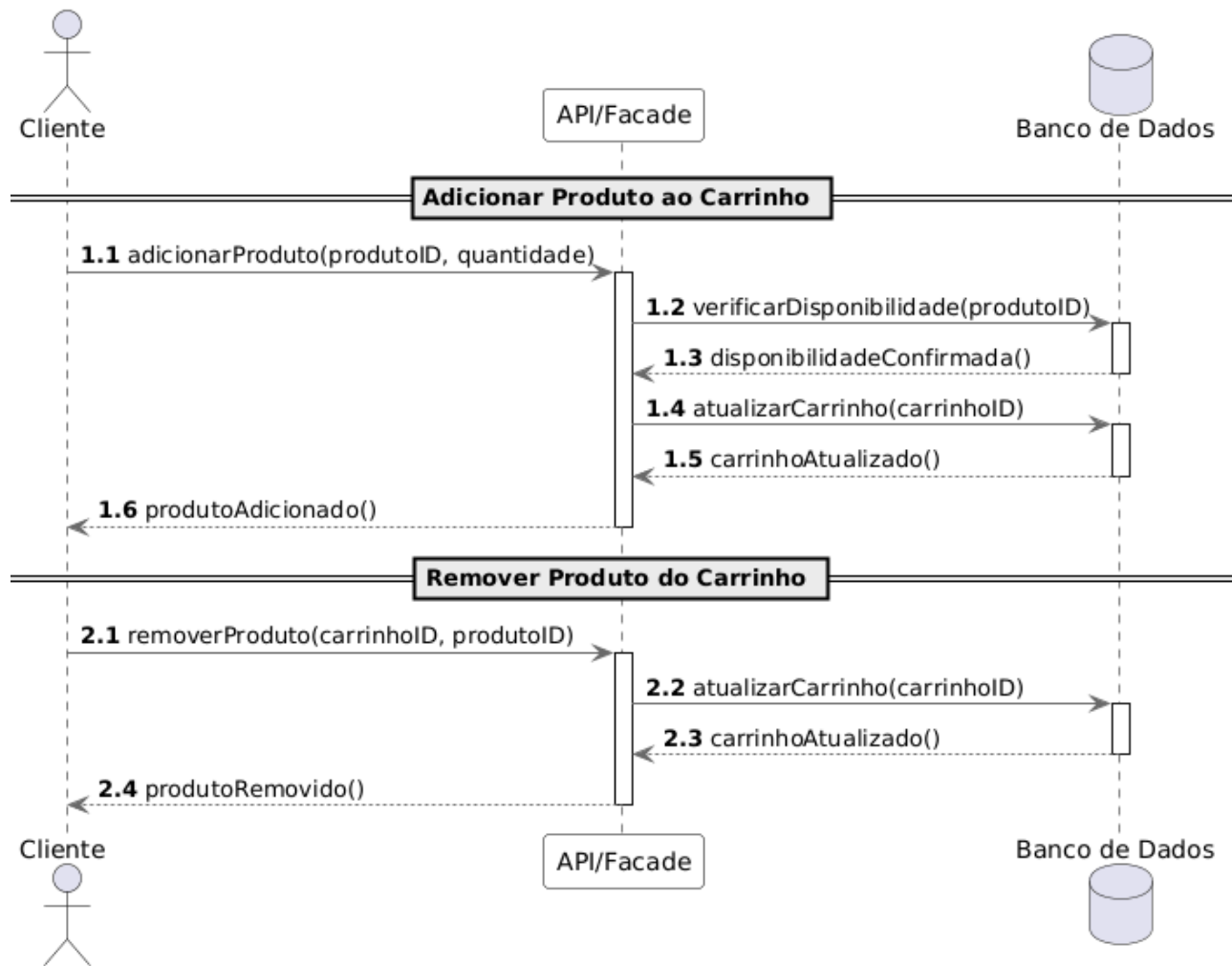
Contrato	CO-03: Atualizar Dados da Conta
Operação	atualizarDadosConta(dadosUsuario)
Referências cruzadas	Caso de Uso: Gerenciar Conta (Cliente ou Vendedor)
Pré-condições	O Usuario deve estar autenticado no sistema e seu registro deve existir no Banco de Dados.
Pós-condições	- A instância correspondente de Usuario foi recuperada do Banco de Dados. - Os atributos da instância foram substituídos pelos novos dadosUsuario. - O estado atualizado do Usuario foi salvo no Banco de Dados.



Contrato	CO-04: Buscar Produtos no Catálogo
Operação	buscarProdutos(filtros)
Referências cruzadas	Caso de Uso: Navegar pelo Catálogo de Jogos
Pré-condições	O sistema deve estar online e o catálogo de produtos acessível.
Pós-condições	- Uma consulta é realizada no Banco de Dados aplicando os filtros especificados. - Uma lista de produtos compatíveis é retornada para exibição ao Cliente. - O estado do sistema permanece inalterado (operação estritamente de leitura).

Contrato	CO-05: Visualizar Detalhes do Produto
Operação	visualizarDetalhes(produtoID)
Referências cruzadas	Caso de Uso: Navegar pelo Catálogo de Jogos

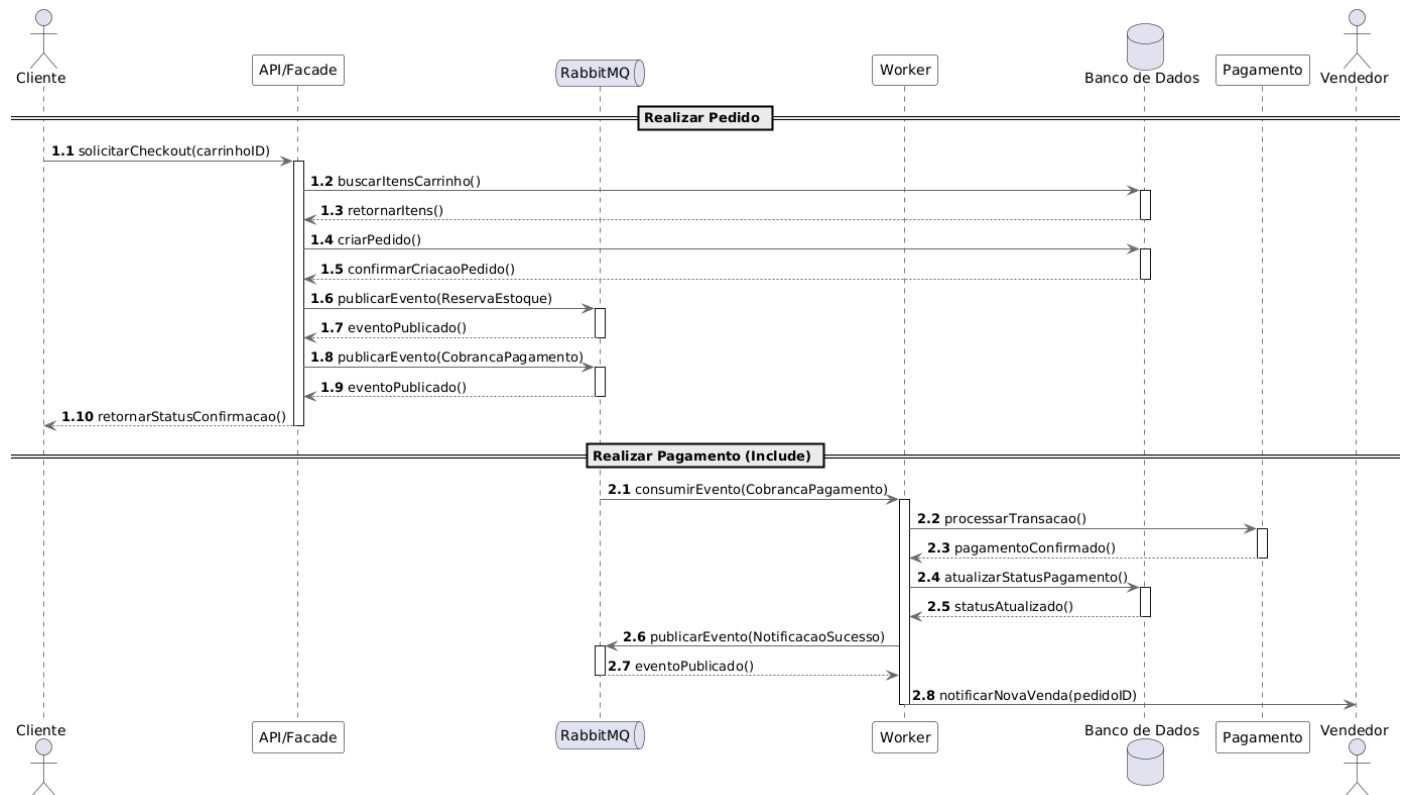
Pré-condições	O produtoID selecionado deve ser válido e existir ativamente no Banco de Dados.
Pós-condições	- Os dados completos e específicos do produto correspondente ao produtoID são recuperados do Banco de Dados. - O estado do sistema permanece inalterado (operação estritamente de leitura).



Contrato	CO-06: Adicionar Produto ao Carrinho
Operação	<code>adicionarProduto(produtoID, quantidade)</code>
Referências cruzadas	Caso de Uso: Gerenciar Carrinho de Compras
Pré-condições	O produtoID deve ser válido no sistema e o banco de dados deve confirmar a disponibilidade de estoque para a quantidade solicitada.
Pós-condições	- A disponibilidade do produto foi confirmada no Banco de Dados .

	- A instância do carrinho foi atualizada para incluir a quantidade do produtoID . - O novo estado do carrinho foi persistido no Banco de Dados.
--	--

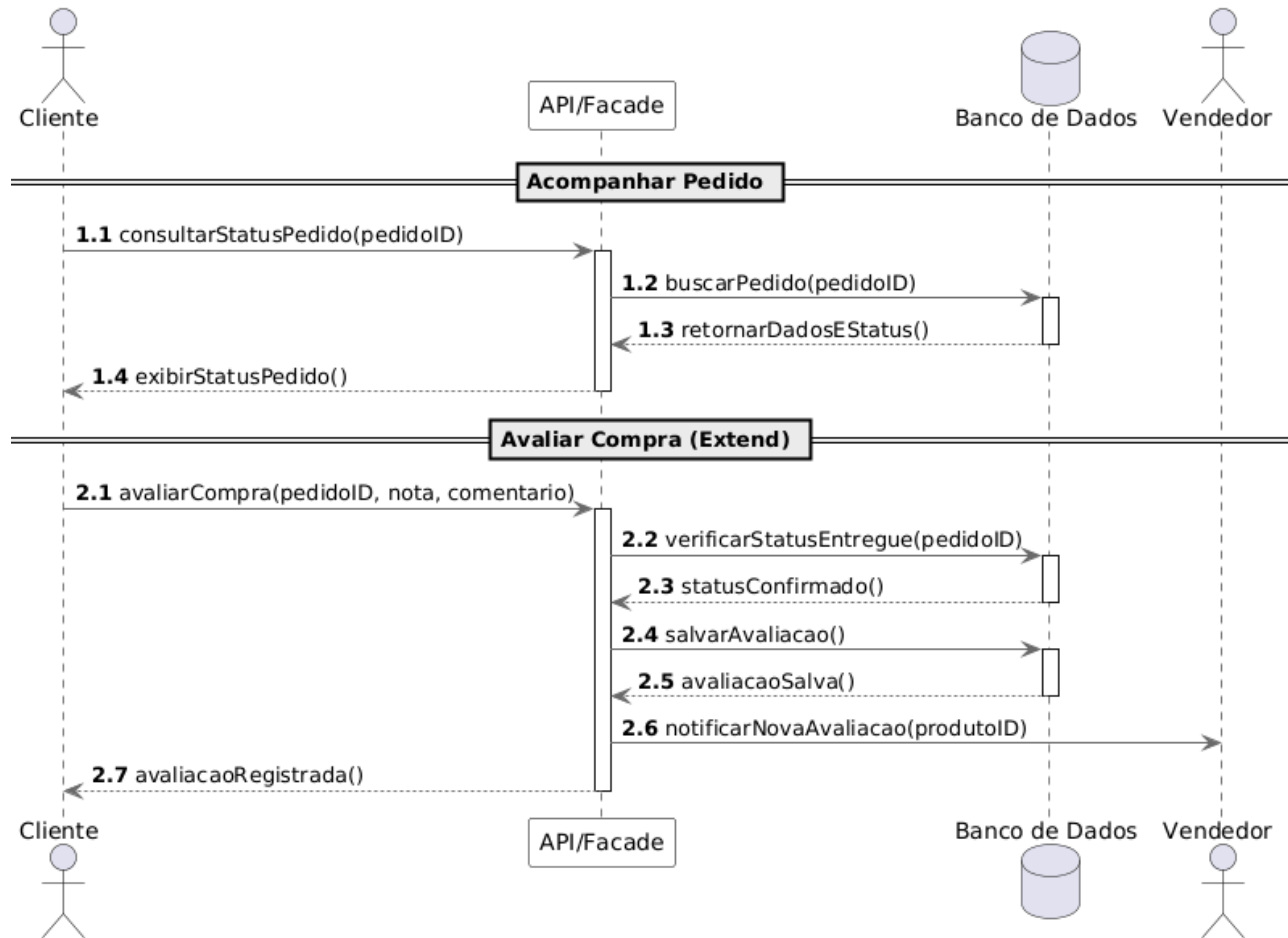
Contrato	CO-07: Remover Produto do Carrinho
Operação	removerProduto(carrinhoID, produtoID)
Referências cruzadas	Caso de Uso: Gerenciar Carrinho de Compras
Pré-condições	O produtoID deve estar presente no carrinho identificado pelo carrinhoID.
Pós-condições	- O produto correspondente ao produtoID foi retirado da lista de itens do carrinho. - O registro atualizado do carrinho foi salvo no Banco de Dados refletindo a remoção.



Contrato	CO-08: Solicitar Checkout
Operação	solicitarCheckout(carrinhoID)
Referências cruzadas	Caso de Uso: Realizar Pedido

Pré-condições	O carrinhoID deve ser válido e conter pelo menos um item associado a ele no Banco de Dados.
Pós-condições	- Uma nova instância de Pedido foi criada com base nos itens do carrinho. - O Pedido foi persistido no Banco de Dados (aguardando pagamento). - Um evento de ReservaEstoque foi publicado no mensageiro (RabbitMQ). - Um evento de CobrancaPagamento foi publicado no mensageiro (RabbitMQ).

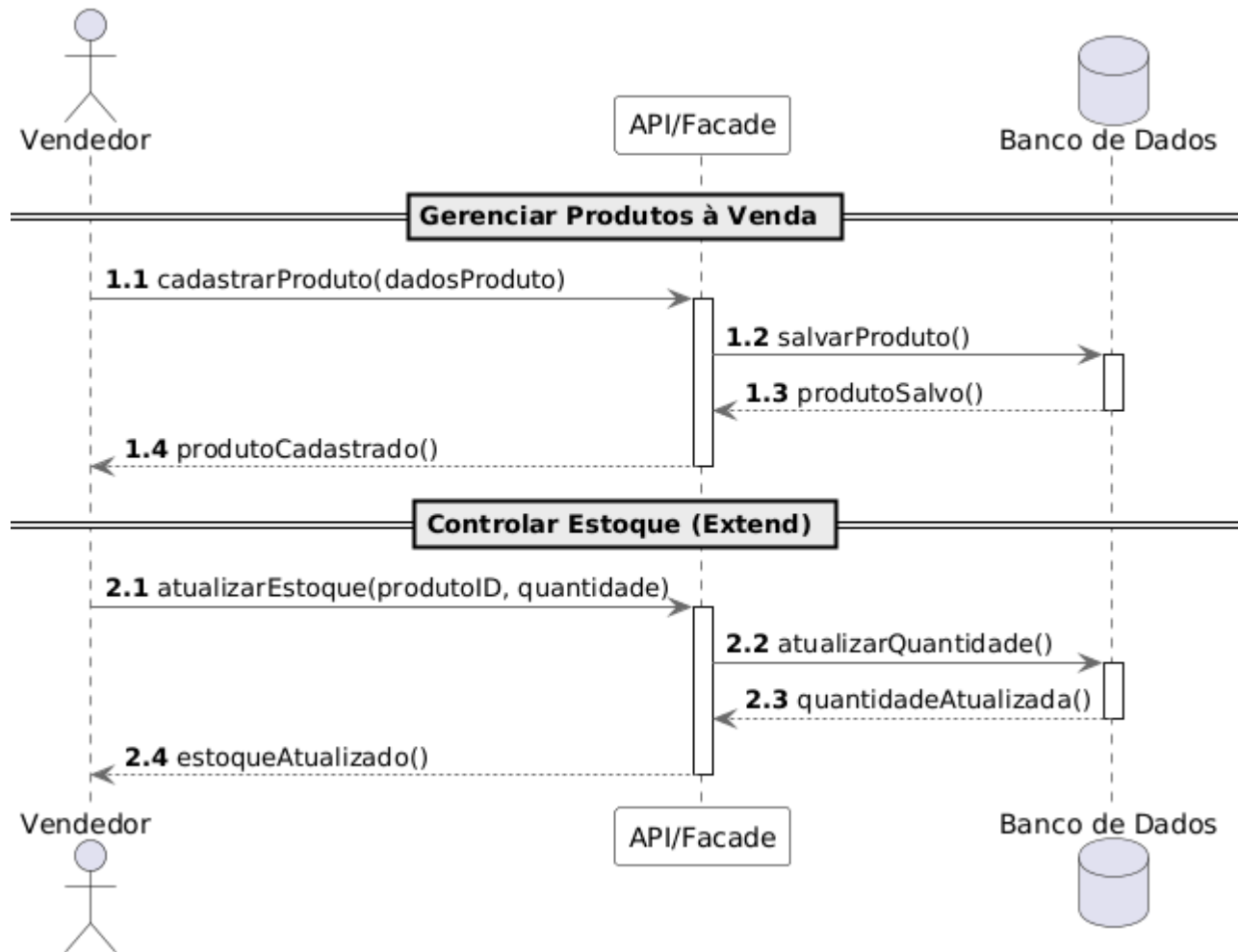
Contrato	CO-09: Processar Pagamento
Operação	processarTransacao()
Referências cruzadas	Caso de Uso: Realizar Pedido (Include: Realizar Pagamento)
Pré-condições	Um evento de cobrança válido deve ter sido consumido da fila pelo Worker e o pedido correspondente deve estar aguardando pagamento.
Pós-condições	- A transação financeira foi confirmada pelo sistema externo de Pagamento. - O status do Pedido foi atualizado no Banco de Dados para refletir a aprovação. - Um evento de NotificacaoSucesso foi publicado no mensageiro (RabbitMQ). - O sistema disparou uma notificação de nova venda para o Vendedor.



Contrato	CO-10: Consultar Status do Pedido
Operação	consultarStatusPedido(pedidoID)
Referências cruzadas	Caso de Uso: Acompanhar Pedido
Pré-condições	O pedidoID deve ser válido, existir no Banco de Dados e estar associado ao Cliente solicitante.
Pós-condições	- Os dados e o status atual do pedido foram recuperados do Banco de Dados. - O estado do sistema permaneceu inalterado (operação estritamente de leitura).

Contrato	CO-11: Avaliar Compra
Operação	avaliarCompra(pedidoID, nota, comentario)
Referências cruzadas	Caso de Uso: Acompanhar Pedido (Extend: Avaliar Compra)
Pré-condições	O Banco de Dados deve confirmar que o status do pedidoID é igual a "ENTREGUE".
Pós-condições	- Uma nova avaliação contendo a nota e o comentário foi criada e vinculada ao pedido/produto. - A avaliação foi salva e persistida no Banco de Dados.

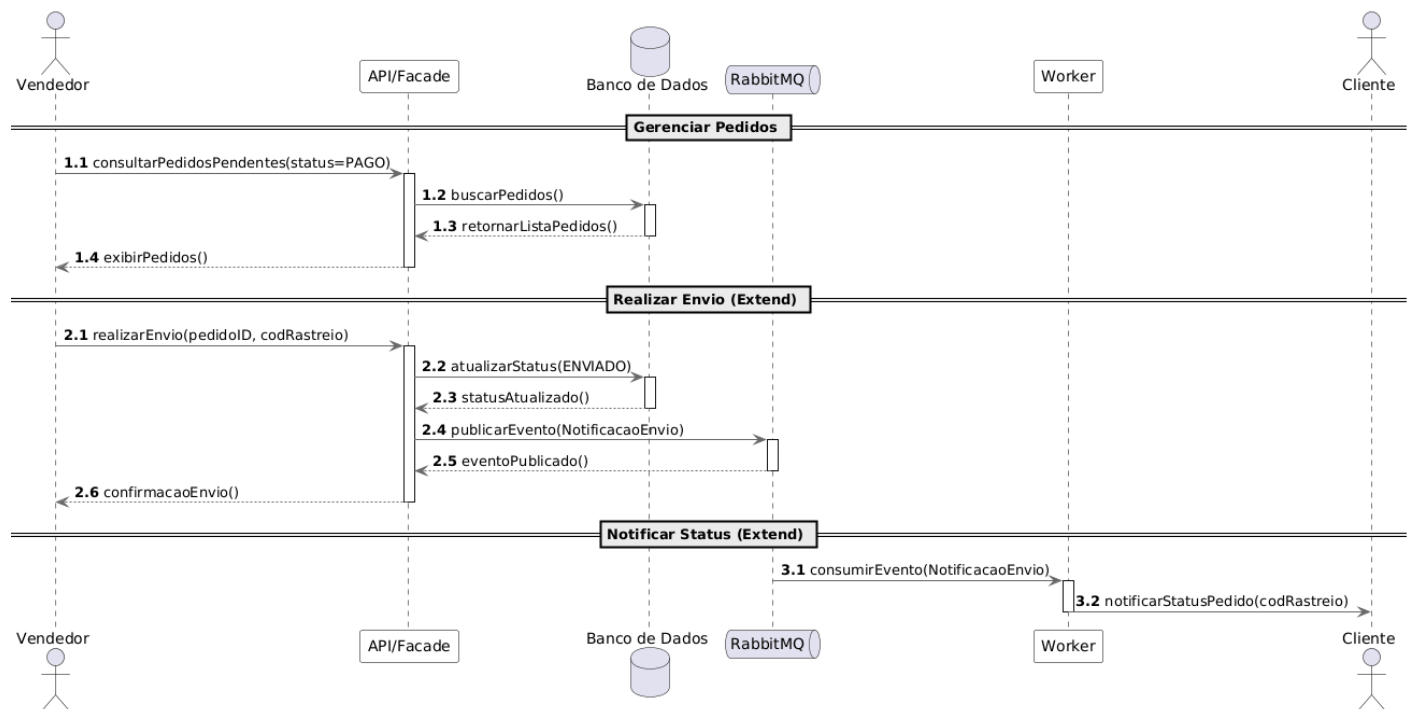
	- O sistema disparou uma notificação para o Vendedor sobre a nova avaliação.
--	--



Contrato	CO-12: Cadastrar Produto
Operação	cadastrarProduto(dadosProduto)
Referências cruzadas	Caso de Uso: Gerenciar Produtos à Venda
Pré-condições	O Vendedor deve estar autenticado no sistema e os dados essenciais do produto devem ser válidos.
Pós-condições	- Uma nova instância de Produto foi criada com base nos dadosProduto. - O produto foi salvo e persistido no Banco de Dados.

Contrato	CO-13: Atualizar Estoque
Operação	atualizarEstoque(produtoID, quantidade)

Referências cruzadas	Caso de Uso: Gerenciar Produtos à Venda (Extend: Controlar Estoque)
Pré-condições	O produtoID deve ser válido, existir no Banco de Dados e pertencer ao catálogo do Vendedor solicitante.
Pós-condições	- A quantidade em estoque do produto correspondente ao produtoID foi atualizada . - O novo valor de quantidade foi persistido no Banco de Dados.



Contrato	CO-14: Consultar Pedidos Pendentes
Operação	consultarPedidosPendentes(status=PAGO)
Referências cruzadas	Caso de Uso: Gerenciar Pedidos
Pré-condições	O Vendedor deve estar autenticado no sistema.
Pós-condições	- A lista de pedidos do Vendedor com o status correspondente foi recuperada do Banco de Dados. - O estado do sistema permaneceu inalterado (operação de leitura).

Contrato	CO-15: Realizar Envio
Operação	realizarEnvio(pedidoID, codRastreio)
Referências cruzadas	Caso de Uso: Gerenciar Pedidos (Extend: Realizar Envio)
Pré-condições	O pedidoID deve ser válido, pertencer ao Vendedor solicitante e estar com o status "PAGO".

Pós-condições	- O status do pedido foi atualizado para "ENVIADO" no Banco de Dados. - Um evento de NotificacaoEnvio foi publicado no mensageiro (RabbitMQ).
----------------------	--

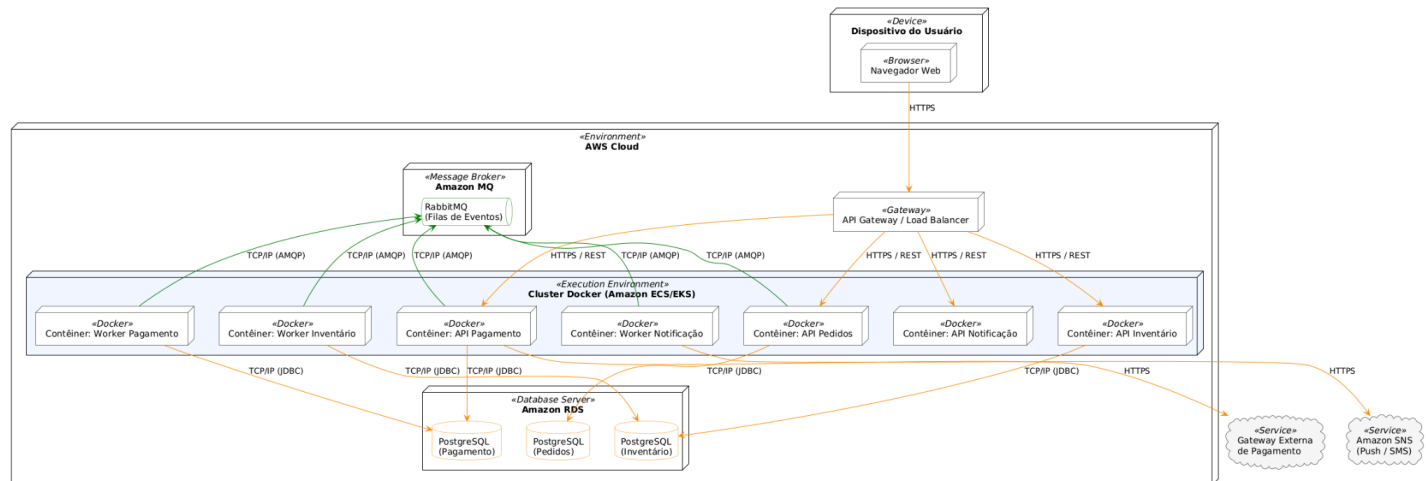
Contrato	CO-16: Notificar Status
Operação	consumirEvento(NotificacaoEnvio)
Referências cruzadas	Caso de Uso: Gerenciar Pedidos (Extend: Notificar Status)
Pré-condições	O Worker de notificação deve ter consumido um evento de envio válido da fila.
Pós-condições	- O Cliente correspondente ao pedido foi notificado sobre a atualização de status e recebeu o código de rastreio.

Contrato	CO-17: Notificar Status
Operação	notificarStatusPedido(codRastreio)
Referências cruzadas	Caso de Uso: Gerenciar Pedidos (Extend: Notificar Status)
Pré-condições	O Worker de notificação deve ter consumido um evento de envio válido da fila.
Pós-condições	- O Cliente correspondente ao pedido foi notificado sobre a atualização de status e recebeu o código de rastreio.

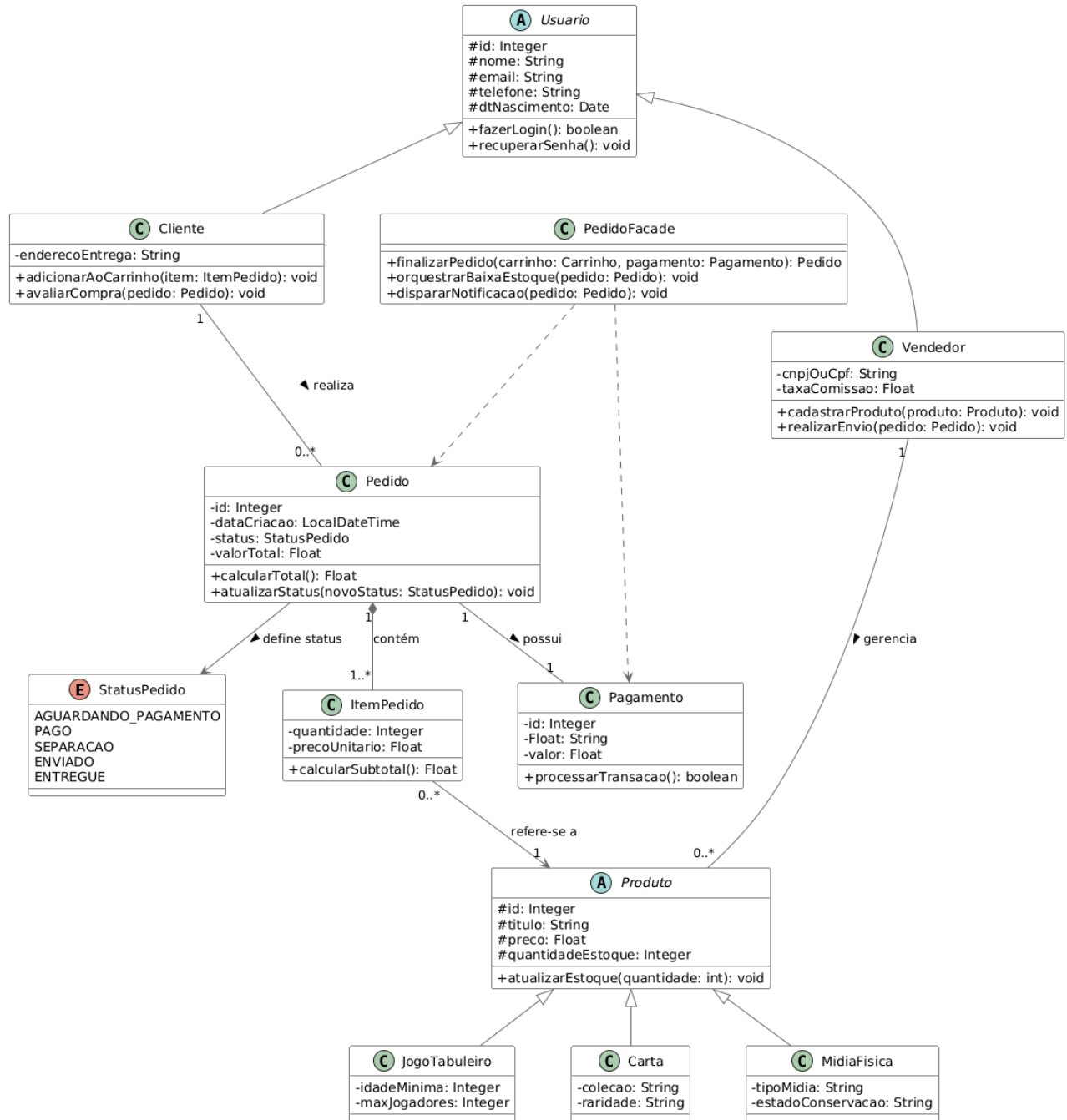
3. Modelos de Projeto

3.1 Arquitetura

- **Componentes**
 - **Catálogo e Inventário:** lida com a listagem de jogos de tabuleiro, cartas, CDs e o controle do estoque dos vendedores.
 - Fila de inventário, RabbitMQ, com worker (para atualizar quantidades assincronamente após uma venda).
 - Dockerizado.
 - Banco dedicado, PostgreSQL.
 - **Pedidos:** lida com as solicitações já feitas pelos clientes e o fechamento do carrinho.
 - Fila de pedidos, RabbitMQ, com worker (para processar a separação e repassar ao pagamento).
 - Dockerizado.
 - Banco dedicado, PostgreSQL.
 - **Pagamento:** lida com o pagamento das compras e a comunicação com a API externa.
 - Fila de pagamento, RabbitMQ, com worker.
 - Dockerizado.
 - Banco dedicado, PostgreSQL.
 - **Notificação:** lida com os avisos de status de envio e confirmações.

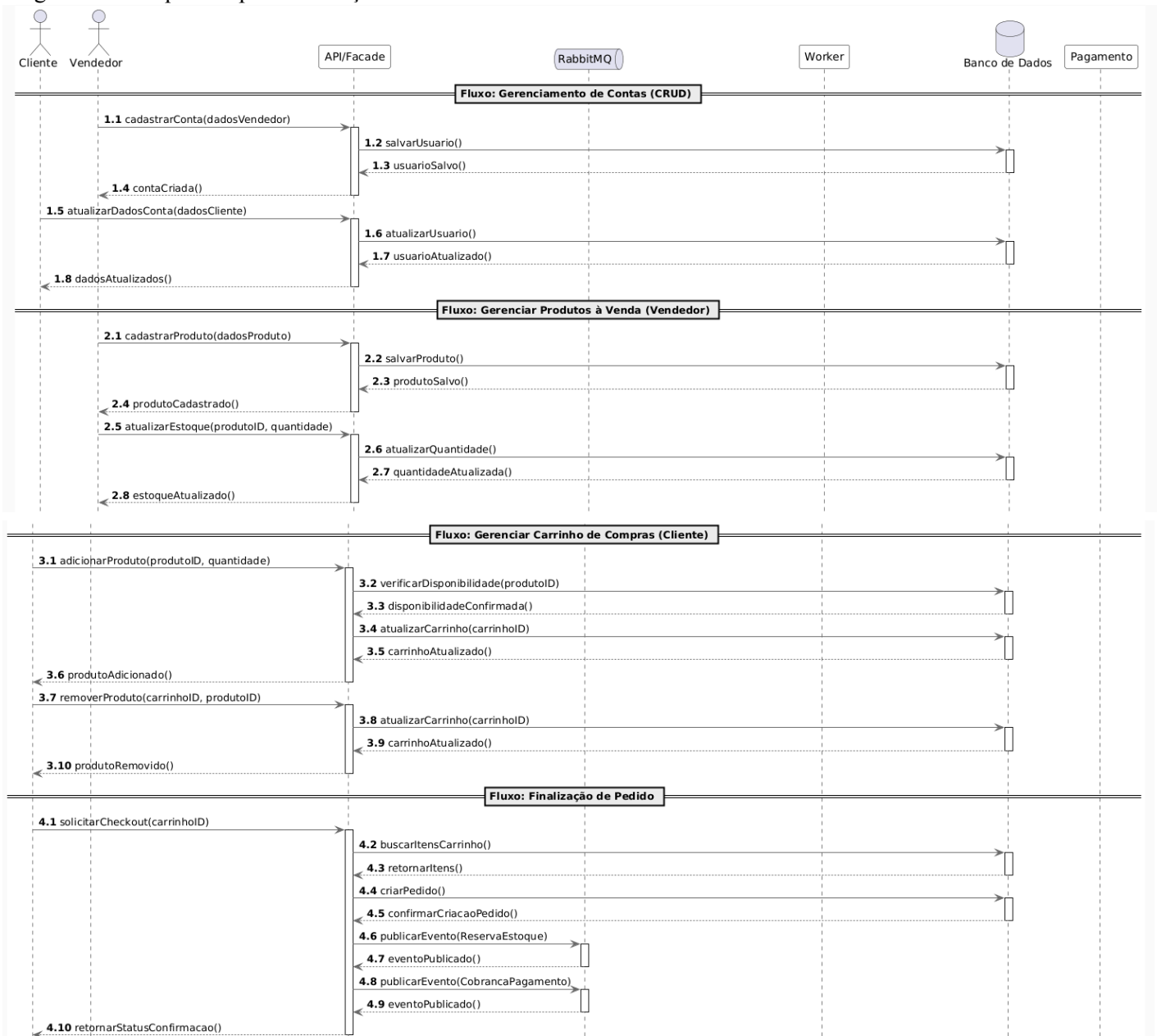


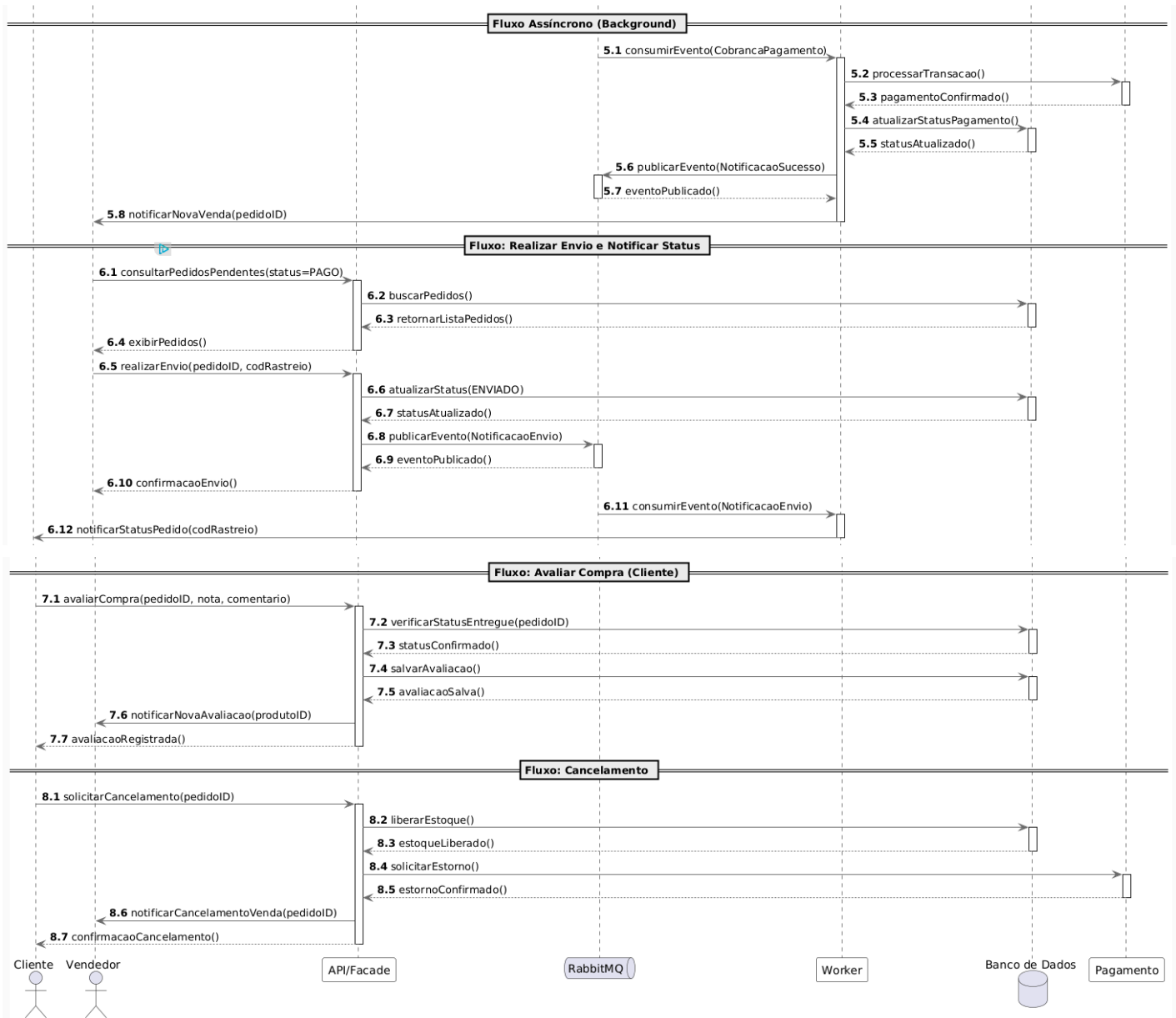
3.3 Diagrama de Classes



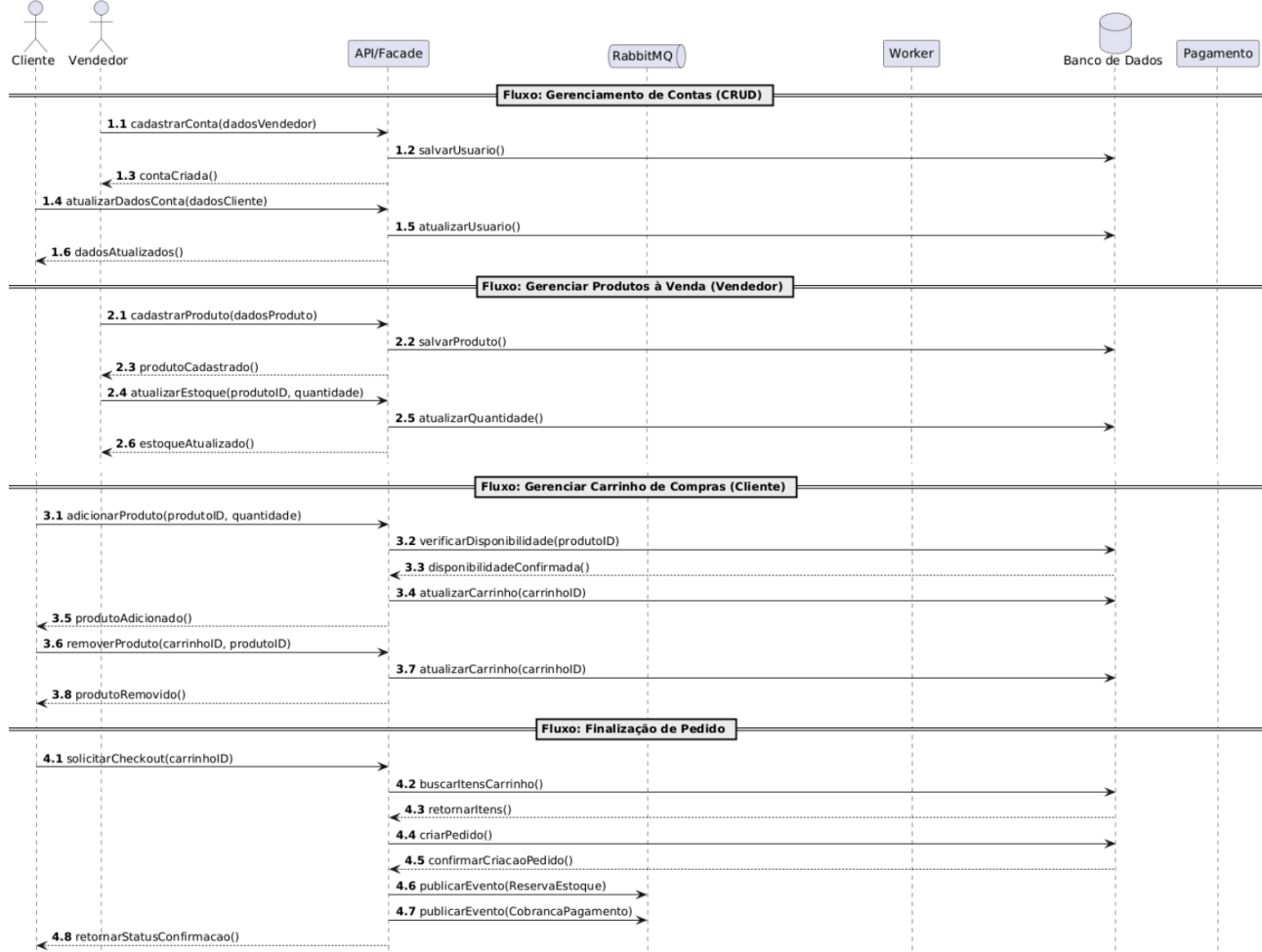
3.4 Diagramas de Sequência

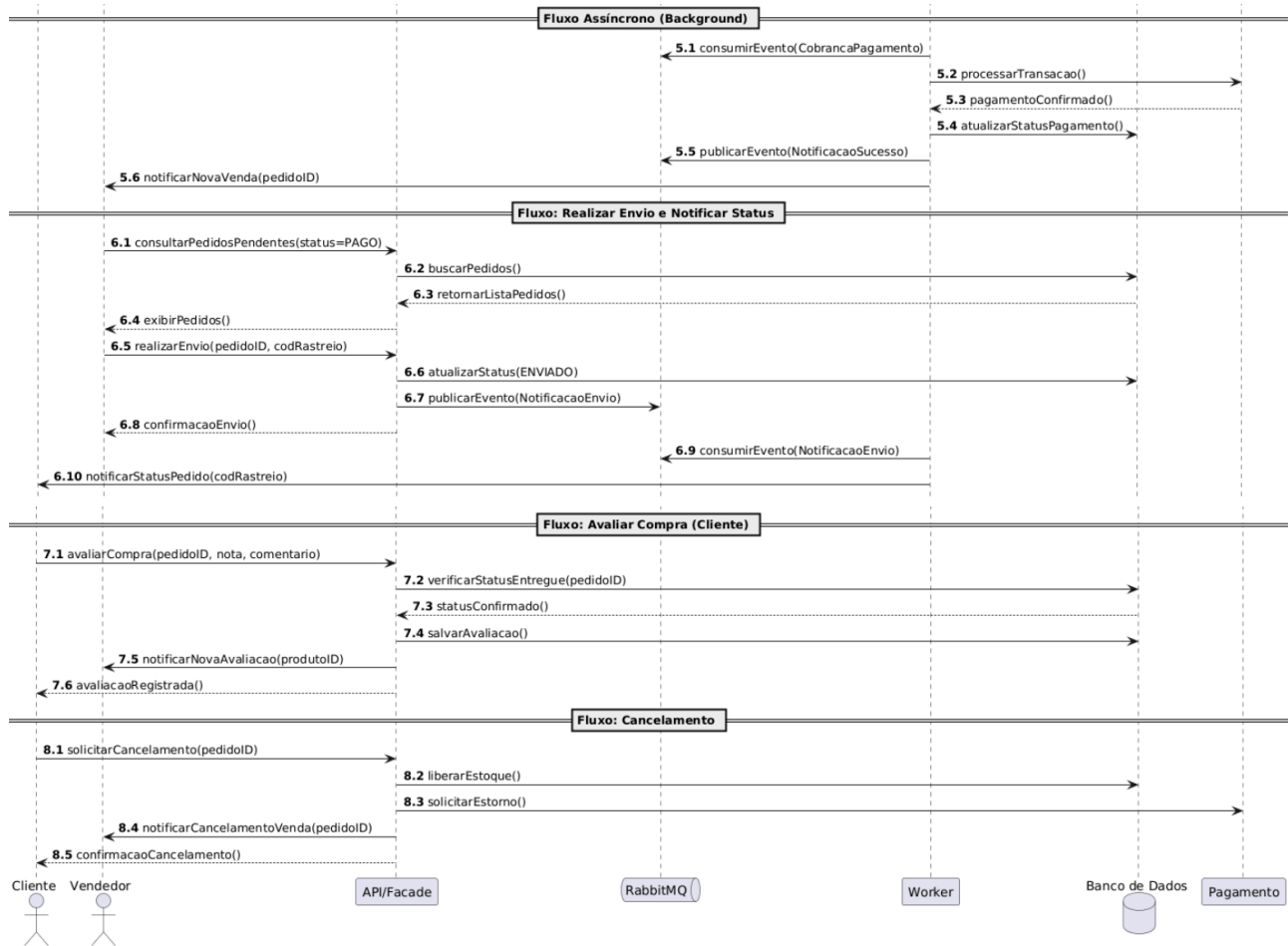
Diagramas de sequência para realização de casos de uso.



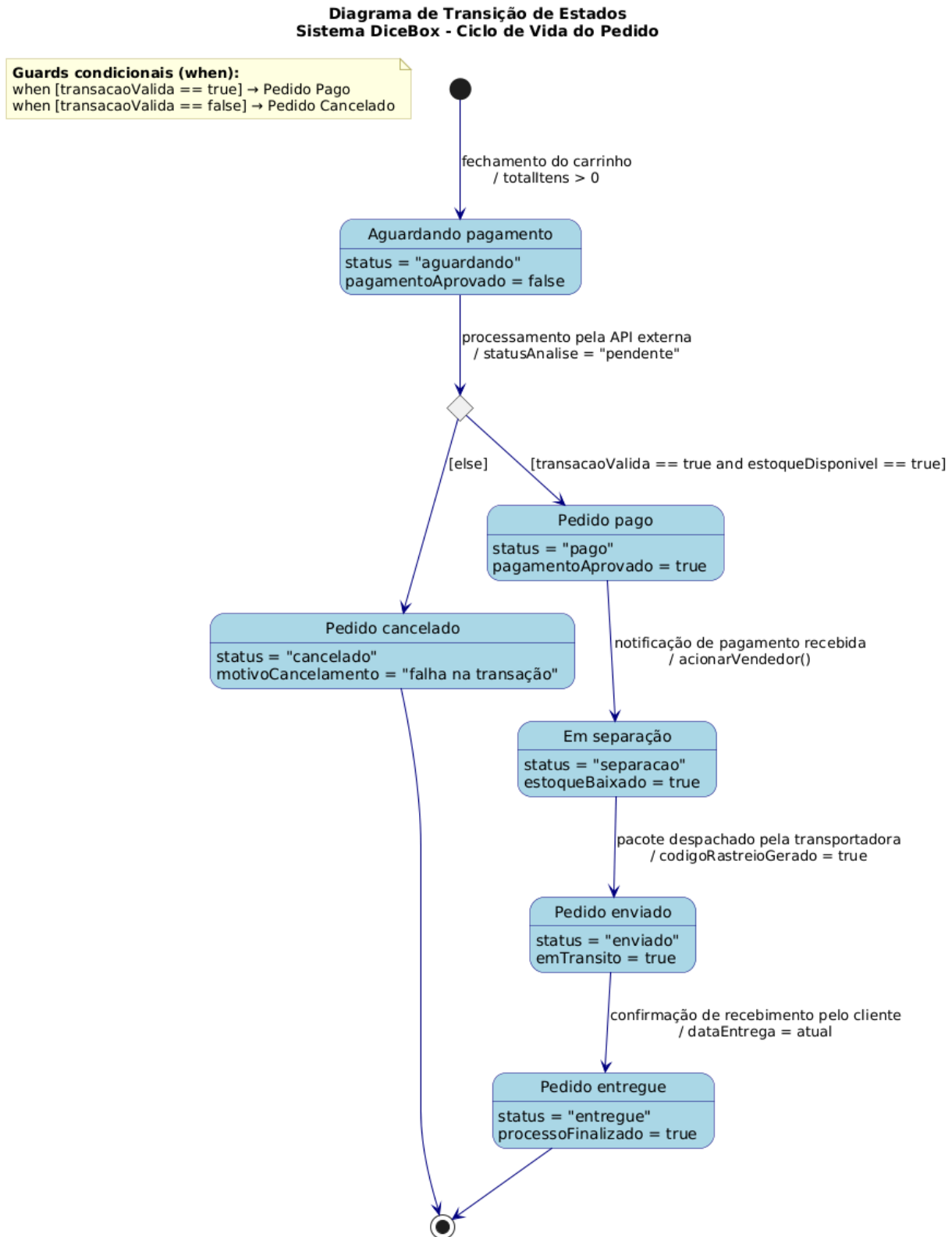


3.5 Diagramas de Comunicação





3.6 Diagramas de Estados



4. Modelos de Dados

O armazenamento de dados do sistema será realizado em bancos de dados relacionais PostgreSQL, com instâncias separadas fisicamente por microserviço. O mapeamento entre o modelo orientado a objetos (aplicação em Java/Micronaut) e o modelo relacional será gerenciado por um framework ORM, JPA/Hibernate.

- **Estratégias de Mapeamento Objeto-Relacional (ORM)**
 - **Mapeamento de Herança (Usuários e Produtos):**
 - **Usuários (Cliente e Vendedor):** Será utilizada a estratégia de herança JOINED (@Inheritance(strategy = InheritanceType.JOINED)). Uma tabela base usuario conterá os dados comuns (ID, nome, email, senha), enquanto tabelas específicas cliente e vendedor armazenarão os atributos exclusivos (endereço de entrega para cliente; CNPJ e taxa para vendedor). Isso evita colunas vazias no banco de dados e garante a normalização.
 - **Produtos (Tabuleiro, Cartas, Mídias Físicas):** Será utilizada a estratégia SINGLE_TABLE (@Inheritance(strategy = InheritanceType.SINGLE_TABLE)). Todas as variações de produtos ficarão em uma única tabela produto. Uma coluna discriminadora chamada tipo_produto ("TABULEIRO", "CARTA", "MIDIA") indicará qual classe o registro representa. Essa estratégia favorece a performance de leitura (queries mais rápidas sem necessidade de JOINS complexos) no microserviço de catálogo.
 - **Mapeamento de Relacionamentos:**
 - **Pedido e Itens (Composição 1:N):** A classe Pedido conterá uma lista de ItemPedido mapeada com @OneToMany(mappedBy = "pedido", cascade = CascadeType.ALL). O ciclo de vida do item está estritamente ligado ao pedido.
 - **Microserviços:** Como o sistema é distribuído, relacionamentos que cruzam as fronteiras dos microserviços não terão chaves estrangeiras físicas rígidas no banco. O mapeamento armazenará apenas o ID da entidade externa, garantindo o isolamento dos dados.

Abaixo está o diagrama lógico representando as entidades, seus atributos, chaves primárias, chaves estrangeiras e a cardinalidade do banco de dados.

